

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture.
(BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

Scenario-Based

Code of Conduct:

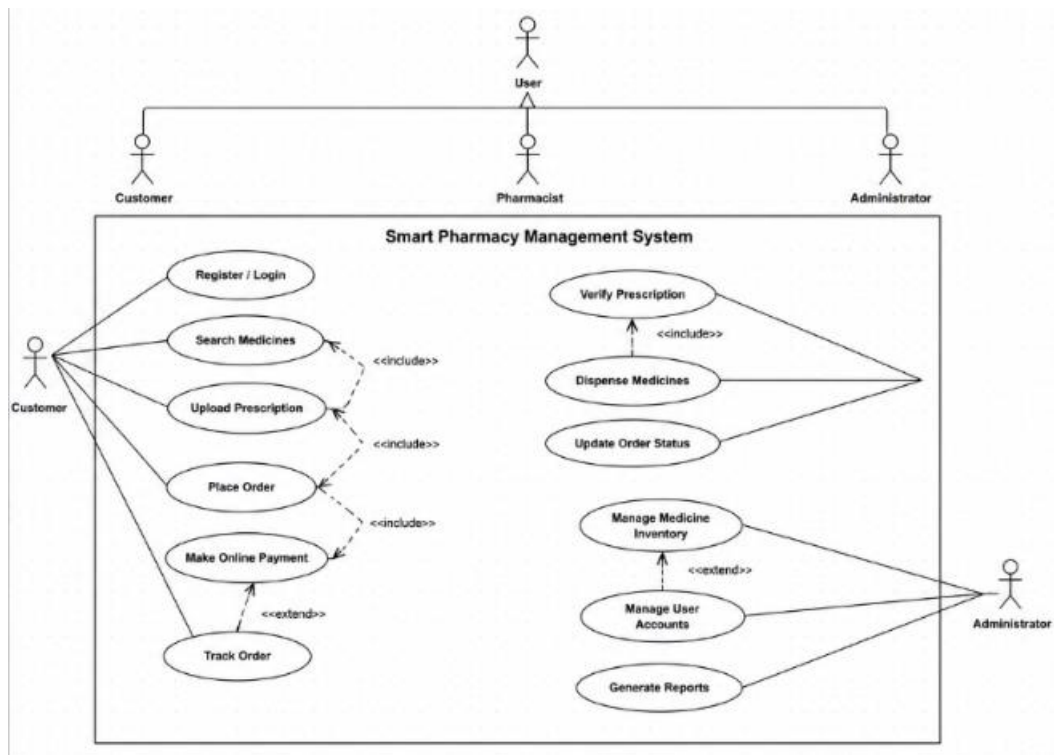
I _____ Delisha A K (192511307)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Smart Pharmacy Management System – Use Case Diagram

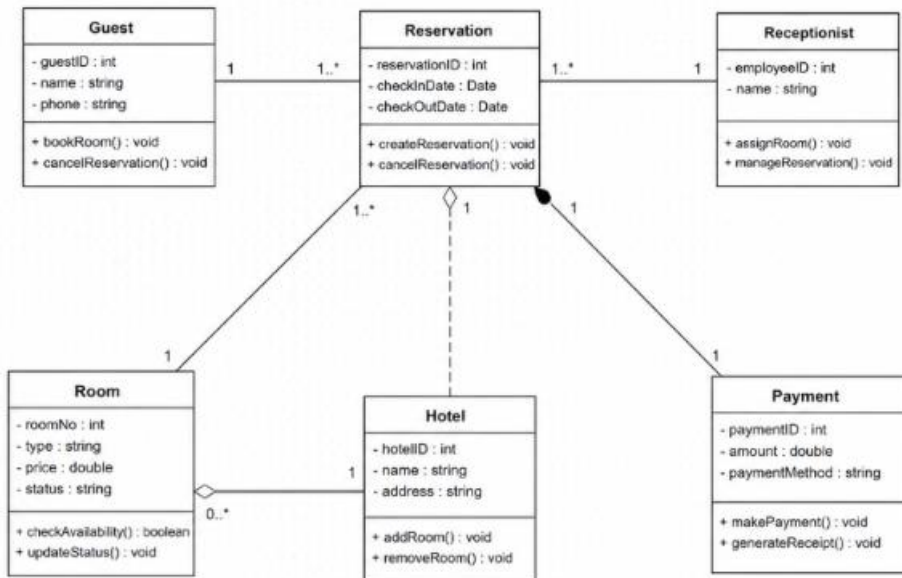
- **Actors:** Customer, Pharmacist, Administrator, Payment Gateway (Secondary).
- **Primary Use Cases:** Register/Login, Search Medicines, Upload Prescription, Place Order, Verify Prescription, Make Online Payment, Manage Inventory.
- **Relationships:**
 - Place Order $\rightarrow$ $\ll include \gg$ $\rightarrow$ Make Online Payment (*Payment is required to order*).
 - Upload Prescription $\rightarrow$ $\ll include \gg$ $\rightarrow$ Register/Login (*Authentication required*).
 - Verify Prescription $\rightarrow$ $\ll extend \gg$ $\rightarrow$ Place Order (*Conditional check for Rx medicines*).
- **Justification & Rationale:** Enforces role-based security by restricting inventory control to administrators and prescription approvals to pharmacists, ensuring regulatory compliance.



2. Smart Hotel Management System – Class Diagram

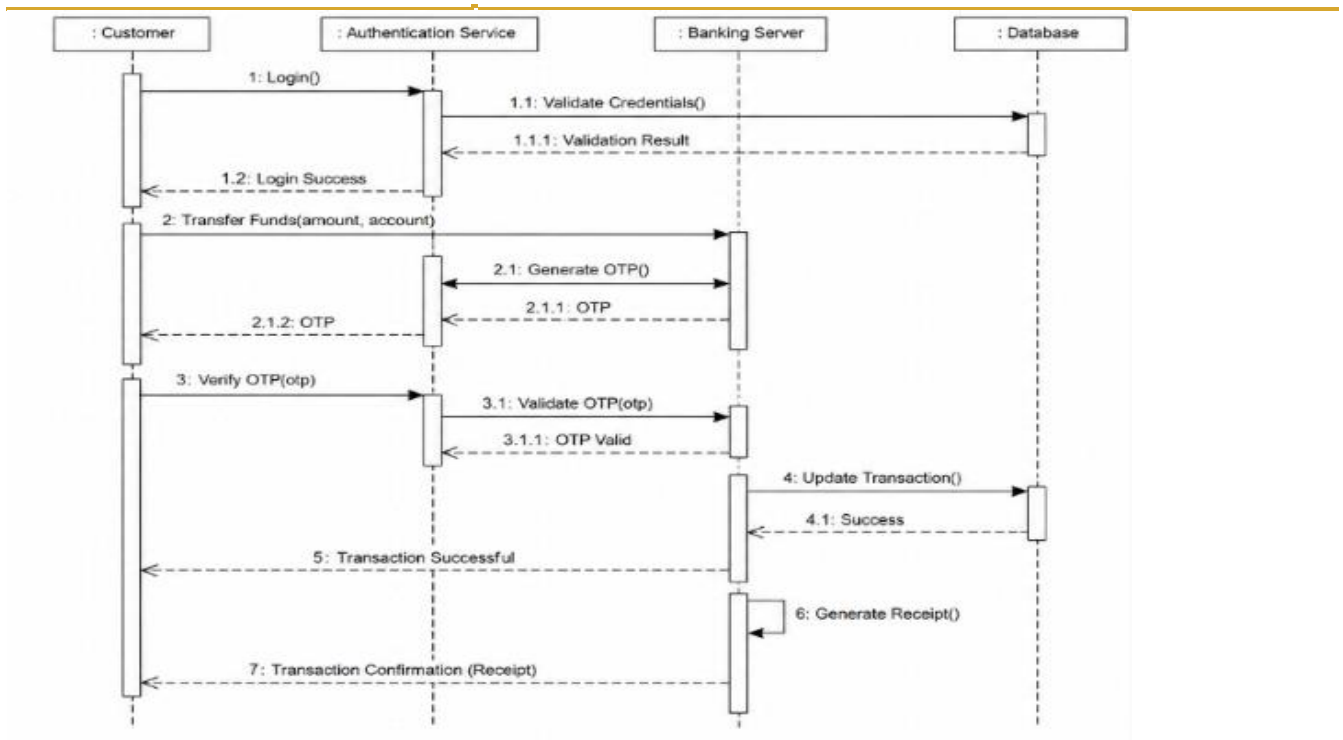
- **Classes & Operations:**
 - Guest: -guestId, -name | +makeReservation()
 - Room: -roomNumber, -price, -status | +checkAvailability()
 - Reservation: -reservationId, -checkInDate | +calculateTotal()
 - Payment: -paymentId, -amount | +processPayment()
 - Receptionist: -employeeId | +assignRoom()
- **Relationships & Multiplicity:**
 - **Composition (\$\blacklozenge\$):** Reservation (1) $\blacklozenge$ Payment (1..*)
Payments cannot exist without a reservation.

- **Aggregation (\$\diamond\$):** Reservation (1..) \$\diamond\$ (1..) Room
(Rooms exist independently of bookings).
- **Association:** Guest (1) \$\rightarrow\$ (0..*) Reservation.
- **Justification & Rationale:** Decouples transactional data from physical room assets, protecting historical booking records even if room details change.



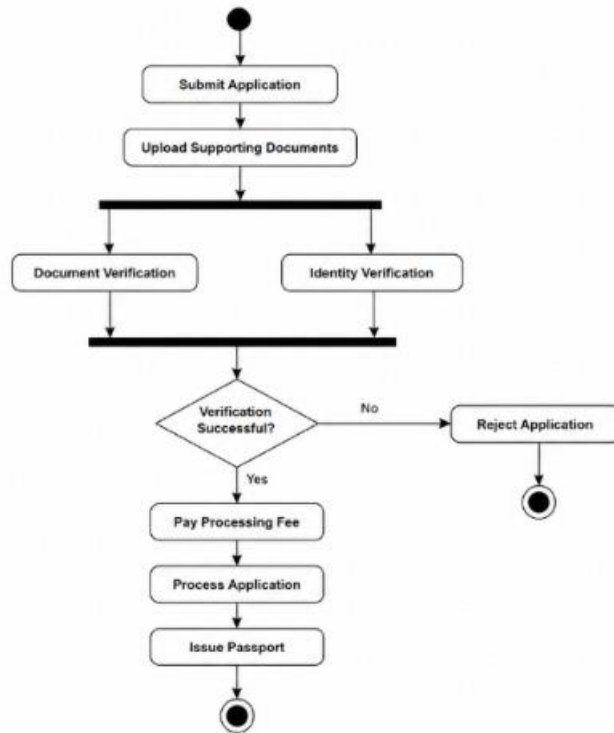
3. Online Banking System – Sequence Diagram

- **Objects**
(**Lifelines**): :Customer, :BankingInterface, :AuthenticationService, :BankingServer, :Database.
- **Message Flow:**
 - login() \$\rightarrow\$:BankingInterface \$\rightarrow\$:AuthenticationService.
 - sendOTP() \$\rightarrow\$:AuthenticationService \$\rightarrow\$:Customer.
 - submitOTP() \$\rightarrow\$:Customer \$\rightarrow\$:AuthenticationService.
 - authSuccess (Dashed Return) \$\rightarrow\$:AuthenticationService \$\dashrightarrow\$:BankingInterface.
 - transferFunds() \$\rightarrow\$:BankingInterface \$\rightarrow\$:BankingServer \$\rightarrow\$:Database.
 - confirmTransaction (Dashed Return) \$\rightarrow\$:Database \$\dashrightarrow\$:BankingServer \$\dashrightarrow\$:Customer.
- **Justification & Rationale:** Synchronous messaging ensures multi-factor authentication (OTP) must succeed before any core banking database updates occur.



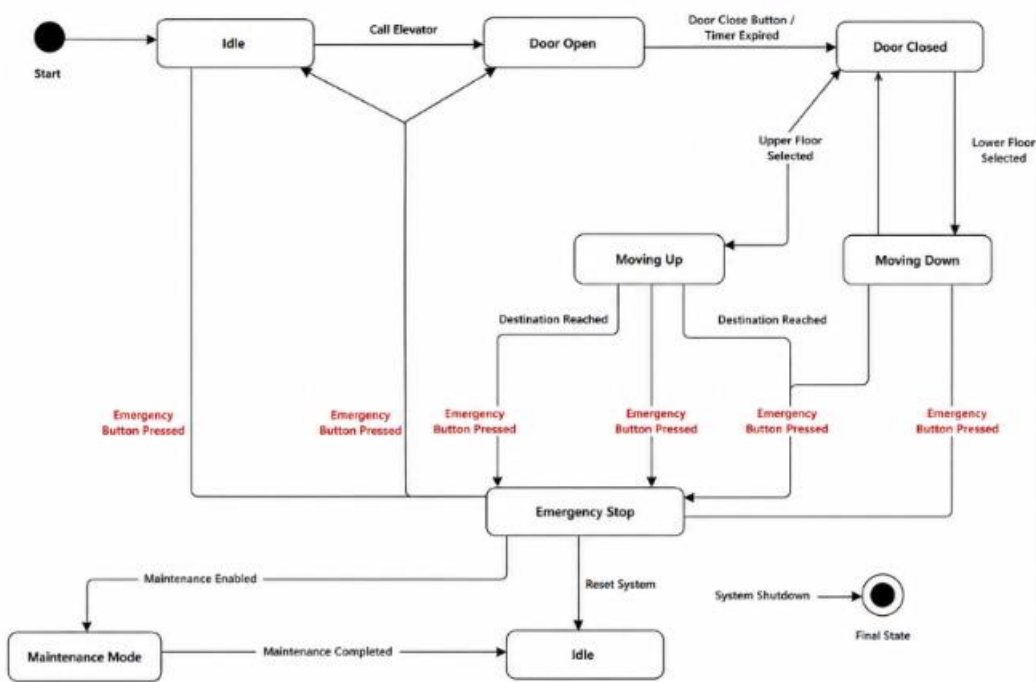
4. Passport Application System – Activity Diagram

- **Key Activities:** Submit Application, Upload Documents, Verify Identity, Process Payment, Print Passport, Dispatch Passport.
- **Decision Nodes:**
 - *Node 1:* Identity Verified? $\rightarrow$ **Yes:** Proceed to payment. | **No:** Reject / Request re-upload.
 - *Node 2:* Payment Success? $\rightarrow$ **Yes:** Proceed to processing. | **No:** Display failure error.
- **Parallel Processing (Fork & Join):** Use a **Fork Bar** after payment to split into Generate Digital Record and Print Physical Passport concurrently, then a **Join Bar** before Dispatch Passport.
- **Justification & Rationale:** Parallel execution optimizes throughput, while decision nodes handle rejection logic without crashing the application flow.



5. Smart Elevator Control System – State Diagram

- **States:** Idle, Door Open, Door Closed, Moving Up, Moving Down, Emergency Stop, Maintenance Mode.
- **Key Transitions & Triggers:**
 - Idle $\rightarrow$ Door Closed (*Trigger: Call Button Pressed*).
 - Door Closed $\rightarrow$ Moving Up/Down (*Trigger: Motor Start [Guard: Target Floor Bar Selected]*).
 - Moving $\rightarrow$ Idle (*Trigger: Target Floor Reached*).
 - **Global Transition:** Any State $\rightarrow$ Emergency Stop (*Trigger: Emergency Switch Activated*).
- **Justification & Rationale:** Global emergency transitions guarantee safety compliance by overriding active operational states immediately upon error detection.



6. Smart Home Automation System – Deployment Diagram

- **Hardware Nodes & Software Artifacts:**

- Smart Sensor Node $\rightarrow$ Deploys «artifact» SensorFirmware.elf
- IoT Home Gateway $\rightarrow$ Deploys «artifact» GatewayMiddleware.jar
- Cloud Application Server $\rightarrow$ Deploys «artifact» SmartHomeCloud.war
- User Mobile Device $\rightarrow$ Deploys «artifact» SmartHomeApp.apk

- **Communication Links & Protocols:**

- Sensor Node $\longleftrightarrow$ Gateway: **Zigbee / Z-Wave**
- Gateway $\longleftrightarrow$ Cloud Server: **MQTT over TLS**
- Mobile Device $\longleftrightarrow$ Cloud Server: **HTTPS / WebSockets**
- Mobile Device $\longleftrightarrow$ Gateway (*Local Fallback*): **Local Wi-Fi (REST)**

- **Justification & Rationale:** Including a local Wi-Fi fallback ensures users maintain direct control over smart appliances even during internet outages.

